Hexagonal vs. Clean architecture

Meisam Alifallhi

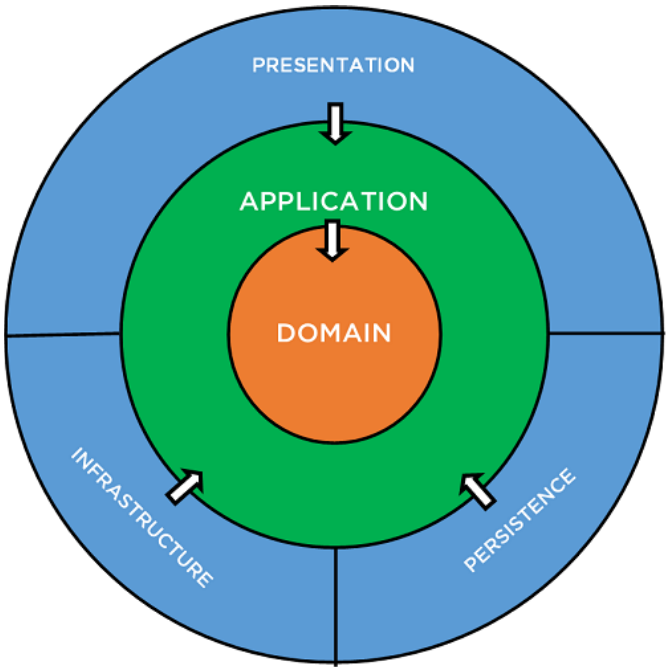
9 min read

Feb 26, 2023

Clean Architecture and Hexagonal Architecture are two popular software architecture patterns used in software development. They have similar goals of creating maintainable, testable, and extensible software systems, but differ in their approach to structuring the project and how they achieve these goals. In this article, we'll explore Clean Architecture and Hexagonal Architecture, compare their differences, pros and cons, and demonstrate how to implement them using C#.

Clean Architecture

Clean Architecture is an architecture pattern that is based on four layers: ***presentation***, ***application***, ***domain***, and ***infrastructure***. The presentation layer is responsible for handling user interactions, the application layer is responsible for handling the business logic, the domain layer is responsible for defining the business entities and their relationships, and the infrastructure layer is responsible for handling external concerns such as databases and web services.



The key principles of Clean Architecture include the independence of the layers, the data flow from the outer layers to the inner layers, and the ability to swap out dependencies without affecting the rest of the system. Clean Architecture emphasizes the separation of concerns to make the codebase easier to maintain, test, and extend.

Project Structure:

The project structure of Clean Architecture follows a structured layout that enables code reusability and testability. In the project structure, you would typically have a project for the presentation layer, a project for the application layer, a project for the domain layer, a project for the infrastructure layer, and a project for the tests. Each layer would have its own set of interfaces and implementations, with the application layer acting as the bridge between the other layers. Here is an example of the project structure of Clean Architecture:

```
MyProject
├── MyProject.Application
│   ├── MyProject.Application.csproj
│   ├── Services
│   ├── UseCases
│   └── ViewModels
├── MyProject.Domain
│   ├── MyProject.Domain.csproj
│   ├── Entities
│   ├── Interfaces
│   └── Services
├── MyProject.Infrastructure
│   ├── MyProject.Infrastructure.csproj
│   ├── Data
│   ├── Repositories
│   └── Services
```

```
├── MyProject.Presentation
│   ├── MyProject.Presentation.csproj
│   ├── Controllers
│   ├── Models
│   └── Views
├── MyProject.Tests
│   ├── MyProject.Tests.csproj
│   ├── ApplicationTests
│   ├── DomainTests
│   └── InfrastructureTests
└── MyProject.sln
```

The project is divided into five separate projects: `MyProject.Application`, `MyProject.Domain`, `MyProject.Infrastructure`, `MyProject.Presentation`, and `MyProject.Tests`.

The `MyProject.Domain` project contains the core business logic and defines the entities and interfaces. This layer is at the center of the architecture, and all other layers depend on it.

The `MyProject.Application` project contains the application logic, including the use cases and services that operate on the domain entities. This layer is responsible for implementing the business rules and application workflows.

The `MyProject.Infrastructure` project contains the implementation details, such as the database access and external services. This layer is responsible for providing the technical details needed to support the application.

The `MyProject.Presentation` project contains the user interface and the presentation logic, such as the web application, the desktop application, or the mobile application.

Finally, the `MyProject.Tests` project contains the tests for the application. This layer is responsible for ensuring that the application functions correctly and that the behavior of the application is consistent with the business requirements.

Implementation:

To implement Clean Architecture in C#, we can use ASP.NET Core. Here is an example of a simple web application that displays a list of books.

```
namespace MyProject.Domain.Entities
{
    public class Book
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public int Year { get; set; }
    }
}
```

```
using System.Collections.Generic;

namespace MyProject.Domain.Interfaces
{
    public interface IBookRepository
    {
        List<Book> GetAll();
    }
}
```

```
using MyProject.Domain.Entities;
using MyProject.Domain.Interfaces;
using System.Collections.Generic;

namespace MyProject.Infrastructure.Repositories
{
    public class BookRepository : IBookRepository
    {
        public List<Book> GetAll()
        {

            return new List<Book>()
            {
                new Book() { Id = 1, Title = "Clean Code", Author = "Robert C. Martin", Year = 2008 },
                new Book() { Id = 2, Title = "The Pragmatic Programmer", Author = "Andrew Hunt and David Thomas", Year = 1999 },
                new Book() { Id = 3, Title = "Design Patterns", Author = "Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides", Year = 1994 }
            };
        }
    }
}
```

```
using MyProject.Domain.Entities;
using MyProject.Domain.Interfaces;
using System.Collections.Generic;

namespace MyProject.Application.Services
{
    public class BookService
    {
        private readonly IBookRepository _bookRepository;

        public BookService(IBookRepository bookRepository)
        {
```

```
            _bookRepository = bookRepository;
        }

        public List<Book> GetAllBooks()
        {
            return _bookRepository.GetAll();
        }
    }
}


using Microsoft.AspNetCore.Mvc;
using MyProject.Application.Services;

namespace MyProject.Presentation.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class BooksController : ControllerBase
    {
        private readonly BookService _bookService;

        public BooksController(BookService bookService)
        {
            _bookService = bookService;
        }

        [HttpGet]
        public IActionResult GetBooks()
        {
            var books = _bookService.GetAllBooks();
            return Ok(books);
        }
    }
}
```

Pros of Clean Architecture:

1. **Separation of concerns**: Clean Architecture enforces a clear separation of concerns between different parts of the system, which makes the codebase more modular and easier to maintain.
2. **Testability:** Clean Architecture makes it easier to test the codebase since each layer is independent of the others, and the dependencies are inverted.
3. **Flexibility:** Clean Architecture makes it easier to swap out dependencies and makes the codebase more flexible and extensible.

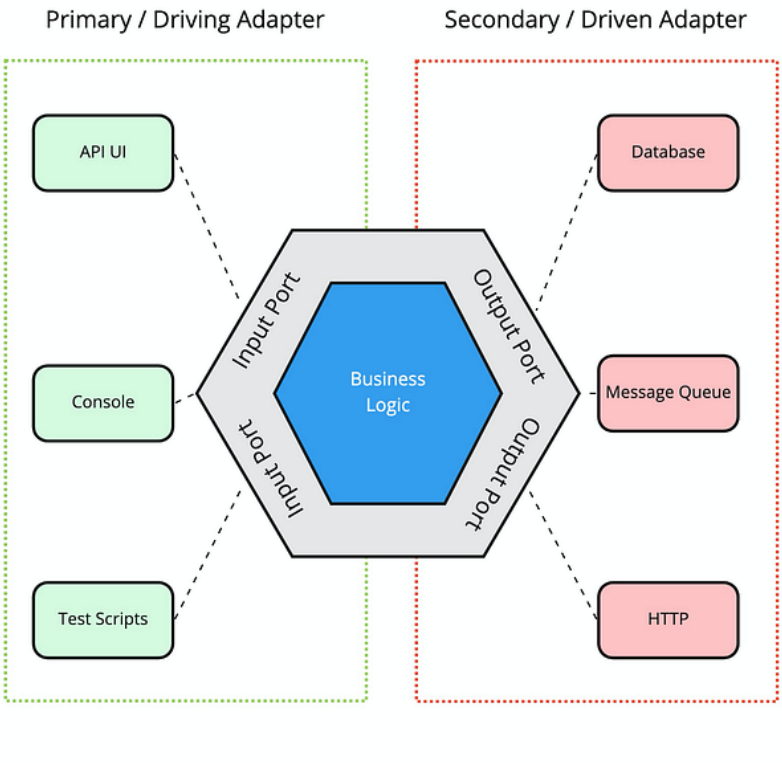
Cons of Clean Architecture:

1. **Complexity:** Clean Architecture can be complex, especially for smaller projects, which may not require such a high level of abstraction.
2. **Learning curve:** Clean Architecture requires developers to have a good understanding of the architecture and the design patterns used, which can increase the learning curve.
3. **Over-engineering:** Clean Architecture can lead to over-engineering, where developers spend too much time on designing the architecture and not enough on delivering functionality.

Hexagonal Architecture

Hexagonal Architecture is a software design pattern that emphasizes the separation of concerns by isolating the application's core logic from the infrastructure components. It does this by dividing the system into two main parts, the **domain** and the **infrastructure**. The domain contains the core business logic of the application, while the infrastructure includes the implementation details of the system.

Press enter or click to view image in full size



The domain layer is where the core business logic of the application is defined. It contains entities, which are objects that represent business concepts or ideas, as well as services, which are responsible for performing operations on these entities. The domain layer is isolated from the infrastructure layer, which means that it does not depend on any specific technology or framework.

Get Meisam Alifallhi's stories in your inbox

Join Medium for free to get updates from this writer.

The infrastructure layer, on the other hand, is responsible for providing the technical details that enable the application to run, such as databases, user interfaces, web services, and other external dependencies. The infrastructure layer contains adapters that communicate with external systems, such as a database, a message queue, or a web service, and translates their requests into a format that the domain layer can understand.

In Hexagonal Architecture, the use of ports and adapters is a key concept that enables the separation of concerns between the domain and infrastructure layers. A port is a contract or an interface that defines a specific functionality or operation that the application provides. An adapter, also known as a driver, implements the port and handles the communication with the external system.

The ports and adapters pattern allows the application to be flexible and easily maintainable. Since the domain layer is isolated from the infrastructure layer, changes to the infrastructure layer can be made without affecting the domain layer. This makes it easy to swap out adapters or to add new adapters for different external systems.

Project Structure:

The project structure of Hexagonal Architecture is divided into four separate projects: domain, application, adapters, and tests. Each of these layers would contain their own set of interfaces and implementations, with the application layer acting as the bridge between the domain and adapter layers.

```
MyProject
├── MyProject.Domain
│   ├── MyProject.Domain.csproj
│   ├── Entities
│   ├── Interfaces
│   └── Services
├── MyProject.Application
│   ├── MyProject.Application.csproj
│   ├── Services
│   └── Interfaces
├── MyProject.Adapters
│   ├── MyProject.Adapters.csproj
│   ├── RestApi
│   ├── Database
│   └── Interfaces
└── MyProject.Tests
    ├── MyProject.Tests.csproj
    ├── ApplicationTests
    └── DomainTests
```

The `MyProject.Domain` project contains the domain logic, which includes the entities, interfaces, and services.

The `MyProject.Application` project contains the application logic, which includes the services and interfaces.

The `MyProject.Adapters` project contains the adapters, which include the rest api, database, and interfaces.

The `MyProject.Tests` project contains the tests for the application and domain layers.

Implementation:

```
namespace MyProject.Domain.Entities
{
    public class Book
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public string Author { get; set; }
        public int Year { get; set; }
    }
}
```

```
using System.Collections.Generic;

namespace MyProject.Adapters.Interfaces
{
    public interface IBookRepository
    {
        List<Book> GetAll();
    }
}
```

```
using MyProject.Domain.Entities;
using MyProject.Adapters.Interfaces;
using System.Collections.Generic;

namespace MyProject.Adapters.Database
{
    public class BookRepository : IBookRepository
    {
        public List<Book> GetAll()
        {

            return new List<Book>()
            {
                new Book() { Id = 1, Title = "Clean Code", Author = "Robert C. Martin", Year = 2008 },
                new Book() { Id = 2, Title = "The Pragmatic Programmer", Author = "Andrew Hunt and David Thomas", Year = 1999 },
                new Book() { Id = 3, Title = "Design Patterns", Author = "Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides", Year = 1994 }
            };
        }
    }
}
```

```
using MyProject.Domain.Entities;
using System.Collections.Generic;

namespace MyProject.Application.Interfaces
{
    public interface IBookService
    {
        List<Book> GetAllBooks();
    }
}
```

```
using MyProject.Domain.Entities;
using MyProject.Adapters.Interfaces;
using MyProject.Application.Interfaces;
using System.Collections.Generic;

namespace MyProject.Application.Services
{
    public class BookService : IBookService
    {
        private readonly IBookRepository _bookRepository;

        public BookService(IBookRepository bookRepository)
        {
            _bookRepository = bookRepository;
        }

        public List<Book> GetAllBooks()
        {
            return _bookRepository.GetAll();
        }
    }
}
```

```
using Microsoft.AspNetCore.Mvc;
using MyProject.Application.Interfaces;

namespace MyProject.Adapters.RestApi.Controllers
{
```

```
    [Route("api/[controller]")]
    [ApiController]
    public class BooksController : ControllerBase
    {
        private readonly IBookService _bookService;

        public BooksController(IBookService bookService)
        {
            _bookService = bookService;
        }

        [HttpGet]
        public IActionResult GetBooks()
        {
            var books = _bookService.GetAllBooks();
            return Ok(books);
        }
    }
}
```

Pros of Hexagonal Architecture:

1. **Separation of concerns:** Hexagonal Architecture separates the core business logic from the implementation details, which makes the codebase more modular and easier to maintain.
2. **Flexibility:** Hexagonal Architecture makes it easier to swap out adapters and makes the codebase more flexible and extensible.
3. **Testability:** Hexagonal Architecture makes it easier to test the codebase since the core business logic is isolated from the implementation details.

Cons of Hexagonal Architecture:

1. **Complexity:** Hexagonal Architecture can be complex, especially for larger projects, which may require more granular separation of concerns.
2. **Over-engineering:** Hexagonal Architecture can lead to over-engineering, where developers spend too much time on designing the architecture and not enough on delivering functionality.
3. **Additional overhead:** Hexagonal Architecture introduces additional overhead in terms of the number of interfaces and classes required, which can increase the complexity of the codebase.

In conclusion, both Clean Architecture and Hexagonal Architecture are software architecture patterns that aim to create maintainable, testable, and extensible software systems. Although both patterns share similar goals, they differ in their approach to structuring the project and how they achieve these goals.

Clean Architecture focuses on the independence of the layers, the data flow from the outer layers to the inner layers, and the ability to swap out dependencies without affecting the rest of the system.

On the other hand, Hexagonal Architecture is based on the separation of concerns between the domain and the infrastructure. This approach uses ports and adapters to define the functionality of the application and allows for the implementation details to be easily swapped out without affecting the rest of the system. Hexagonal Architecture focuses on the separation of concerns between the domain and the infrastructure, the use of ports to define the functionality of the application, and the ability to swap out adapters for different external systems.

Both Clean Architecture and Hexagonal Architecture have their strengths and weaknesses, and the choice of which pattern to use depends on the specific requirements of the project. Clean Architecture may be more suitable for projects with complex business rules, while Hexagonal Architecture may be more suitable for projects with complex integrations with external systems.

Overall, the adoption of a software architecture pattern such as Clean Architecture or Hexagonal Architecture can bring significant benefits to software development, including improved maintainability, testability, and flexibility. By following the principles of these patterns, developers can create high-quality software systems that are resilient to change and adaptable to evolving business requirements.